1

דו"ח מכין למעבדה 5
Scalar Pipelined RISC-V CPU Architecture

בן 5 שלבים Pipeline ומימוש Single-Cycle מימוש - RV32IM מעבד

שם הקורס מעבדת ארכיטקטורת מעבדים מתקדמת ומאיצי חומרה
מספר קורס 361.1.4693
מגישים אדר שפירא 209580208
יהונתן דדכה 211468582
תאריך הגשה 23.08.2026

2

1. מטרות המעבדה ורקע תיאורטי

מטרת מעבדה זו היא לתכנן, לסנתז ולנתח מעבד תואם RISC-V, ולהעלות אותו ממימוש single-cycle למימוש scalar pipelined בן חמישה שלבים. העבודה התמקדה בארבעה נושאים:

• תכנון, סינתזה וניתוח של מעבד תואם RISC-V.
• מימוש מיקרו־ארכיטקטורת pipeline הכוללת יחידת forwarding מלאה ויחידת גילוי hazards בגישת
.combinational check

• ההבדל בין CPU ל- MCU: הליבה שתכננו היא מעבד בלי היקפיים, בלי פסיקות ובלי בקרי קלט/פלט. יש בה
מסלול נתונים, יחידת בקרה ושני זיכרונות צמודים בלבד.
• מבנה הזיכרון ב- FPGA: את ה- ITCM ואת ה- DTCM מימשנו באמצעות בלוקי M9K המשובצים ברכיב.
ארכיטקטורת RISC-V היא ארכיטקטורת Harvard: זיכרון ההוראות וזיכרון הנתונים מופרדים לחלוטין, ולכל אחד מהם אפיק משלו. ההפרדה מעלה את קצב התפוקה (throughput(, מפני שאפשר לשלוף הוראה ולגשת לנתון באותו
מחזור שעון. היא גם מפשטת את הלוגיקה של שלב ה- MEM ב- pipeline, שכן אין תחרות (structural hazard (בין
שליפת הוראה לבין גישה לזיכרון נתונים.

2. הגדרת המשימה וארכיטקטורת המערכת

2.1 ארכיטקטורת המערכת הכוללת

המערכת בנויה סביב ליבת RISC-V, המחוברת לשני זיכרונות צמודים: ITCM לאחסון הקוד, ו- DTCM לאחסון
הנתונים. הליבה מקבלת שעון יחיד (clk_i (ואיפוס (rst_i(, כאשר האיפוס מחווט ללחצן KEY0 ומחזיר את ה-PC
להוראה הראשונה בתוכנית.

איור 1: ארכיטקטורת המערכת: ליבת RISC-V עם זיכרונות ITCM ו- DTCM צמודים

הן הישות העליונה והן ליבת ה- RISC-V מומשו כתכן מבני (structural(, כלומר כחיווט מפורש של תת־מודולים ולא כתיאור התנהגותי מונוליתי. כך נשמרת התאמה אחד־לאחד בין קוד ה- VHDL לבין תרשים המיקרו־ארכיטקטורה, ובעזרת ה- RTL Viewer אפשר לאתר בקלות כל בלוק שמופיע בתרשים.

3

2.2 סביבת הפיתוח והכלים

רכיב ערך הערה
כרטיס פיתוח DE2-115 -

Cyclone IV E EP4CE115F29C7 FPGA רכיב
VHDL-2008 קלט Quartus Prime 21.1.0 Lite כלי סינתזה
כלי סימולציה ModelSim מחזור שעון ns 100 ב- testbench
מודל זהב RARS השוואת DTCM.mem מול DTCM.h
מקור שעון על הכרטיס MHz 50 נכנס ל- clk_i
שעון הליבה MHz MCLK 25 נגזר ב- PLL: 50 × 1/2
טבלה 1: סביבת הפיתוח והכלים ששימשו בעבודה

התכן משתמש בקומפילציה מותנית (cond_compilation_package.vhd (כדי לעבוד בשתי סביבות מאותו קוד מקור. כאשר G_MODELSIM = 1 הזיכרונות נטענים מקבצי hex וה- PLL אינו מיוצר כלל, והשעון מוזן ישירות מה-testbench. כאשר G_MODELSIM = 0 נוצר רכיב ALTPLL הגוזר את שעון הליבה מהמתנד שעל הכרטיס. שאר הפרמטרים בחבילה קובעים את גודל ה- KiB)TCM 8(, את רוחב ה-PC ואת רזולוציית הכתובות) G_WORD_GRANULARITY = True, כלומר כתובת ייחודית לכל מילה(.

4

3. חלק 1 - מיקרו־ארכיטקטורת Single-Cycle RV32IM

Single-Cycle RV32I : נקודת המוצא 3.1

כנקודת מוצא קיבלנו מיקרו־ארכיטקטורת single-cycle התומכת ב- RV32I, ה- ISA הבסיסי הלא־מיוחס. במימוש זה כל הוראה מסיימת את ביצועה במחזור שעון יחיד. ה-PC מספק את כתובת ההוראה, ה- ITCM מחזיר את ההוראה, יחידת הבקרה מפענחת אותה, קובץ האוגרים נקרא, ה- ALU מבצע את החישוב, ה- DTCM נגיש לקריאה או לכתיבה, והתוצאה נכתבת בחזרה לקובץ האוגרים, הכול בתוך אותו מחזור.

איור 2: מיקרו־ארכיטקטורת Single-Cycle RV32I: נקודת המוצא של המשימה

המימוש מחולק לחמישה תת־מודולים פונקציונליים המהווים גם את הבסיס לחלוקה לשלבי ה- pipeline בחלק 2: .DMEMORY-ו EXECUTE ,CONTROL ,IDECODE ,IFETCH

5

איור 3: חלוקת המיקרו־ארכיטקטורה לתת־מודולים) submodules)

ה- DTCM נכתב על השעון ההפוך)) not(clk). הסיבה: במימוש single-cycle, כתובת הזיכרון היא תוצאת ה- ALU,
שמתייצבת רק בסוף המחזור. כתיבה על העלייה הבאה של השעון הייתה מאחרת את הנתון במחזור שלם. הכתיבה על הירידה מקצה חצי מחזור לייצוב הכתובת וחצי מחזור לפעולת הזיכרון עצמה. להחלטה זו יש השלכה ישירה על ניתוח התזמון: המסלול הקריטי של המימוש הוא מסלול חצי מחזור, ולכן fmax נקבע לפי (tpath · 2)/ 1 ולא לפי tpath / 1.

3.2 הרחבת M: מימוש פקודת mul

משימת חלק 1 היא להעלות את המיקרו־ארכיטקטורה מ- RV32I ל- RV32IM. הרחבת M מגדירה פעולות כפל וחילוק
(MULDIV), אך דרישת המשימה מסתפקת בתמיכה חלקית: פקודת mul בלבד מתוך קבוצת הכפל (mulh ,mul,
.)mulhu ,mulhsu

סמנטיקת הפקודה כפי שהוגדרה במשימה: mul rd, rs1, rs2 מכפילה את חצאי המילה התחתונים (16 ביט) של האוגרים rs1 ו- rs2, וכותבת תוצאה בת 32 ביט לאוגר rd. הפקודה היא מסוג R-type, ולכן כל אותות הבקרה האחרים
) MemtoReg ,RegWrite וכו') זהים לאלו של פעולת ALU רגילה. ההרחבה מתבטאת אך ורק בקידוד ALUOp
חדש ובנתיב נתונים נוסף.

6

איור 4: מיקרו־ארכיטקטורת) Single-Cycle RV32IM(MULDIV partial: הכופל בן 16 הביט משולב לצד ה- ALU, ומרבב חדש בוחר בין שתי התוצאות

3.2.1 אלגוריתם הכפל

האלגוריתם הנדרש מפרק כפל 16×16 לארבעה מכפלים חלקיים בני 8×8, כך שהסינתזה תמפה אותם ישירות על הכופלים המשובצים) embedded 9-bit multipliers)של ה- FPGA במקום לבנות כופל מלוגיקה. הפירוק מתבצע בשני שלבים:

המימוש שלנו מתרגם את המשוואות אחד־לאחד: כל אופרנד מפוצל לשני חצאים בני 8 ביט, ארבעת המכפלים החלקיים מחושבים במקביל, הסכום האמצעי M = P1 + P2 מחושב ברוחב 17 ביט (ביט נוסף לנשא), ולבסוף שלושת האיברים מיושרים לרוחב 32 ביט ומחוברים. כדי לוודא שהמיפוי אכן התבצע כמתוכנן בדקנו את דוח ה- Area, והוא מדווח על 4 יחידות embedded 9-bit multiplier בשני המימושים, בדיוק כמספר המכפלים החלקיים. כלומר הסינתזה מיפתה את הכפל לחומרה הייעודית ולא ללוגיקה.

7

3.2.2 השינויים שנדרשו בקבצי התכן

קובץ השינוי שבוצע

,INST_MUL_MASK ומסכת הביטים INST_MUL הוספת תבנית הזיהוי const_package.vhd
והוספת הקידוד ALU_MUL לרשימת פעולות ה- ALU המשותפת ל-
.EXECUTE-ול CONTROL

ALUOp = והנעת INST_MUL_MASK/INST_MUL זיהוי הפקודה לפי CONTROL.vhd
ALU_MUL. שאר אותות הבקרה נותרו ללא שינוי, שכן mul היא R-type.

MUL16.vhd מודול חדש - הכופל 16×16 ← 32 על בסיס ארבעה מכפלים חלקיים.

EXECUTE.vhd יצירת מופע של MUL16 ובחירת פלטו כ- alu_res כאשר = ALUOp
.ALU_MUL

aux_package.vhd הוספת הצהרת הרכיב MUL16 ושינוי שם הליבה מ- RV32I_CORE ל-
.RV32IM_CORE

טבלה 2: סיכום השינויים במעבר מ- RV32I ל- RV32IM במימוש ה- single-cycle

3.3 הישות העליונה ותרשים בלוקים

הישות העליונה RV32IM_CORE מחווטת את חמשת תת־המודולים ומוציאה החוצה קבוצת אותות ניפוי המשמשים הן את חלון הגלים ב- ModelSim והן את הוולידציה ב- Signal-Tap על הכרטיס: ה-PC, ההוראה, אותות הבקרה
אפיקי ,ALU- נתוני קובץ האוגרים, תוצאת ה ,(Branch_ctrl_o ,MemWrite_ctrl_o ,RegWrite_ctrl_o(המרכזיים
ה- DTCM ומונה מחזורי השעון mclk_cnt_o.

איור 5: הישות העליונה של מימוש ה- RV32IM_CORE :Single-Cycle ואותות הניפוי שלה

במימוש single-cycle כל הוראה נמשכת מחזור אחד בדיוק, ולכן IPC = 1 בהגדרה ומונה מחזורי השעון בסוף הריצה
שווה למספר ההוראות שבוצעו בפועל.

8

3.4 תיאור קבצי ה- HDL

קובץ תיאור

FPGA/ModelSim פרמטרי קומפילציה מותנית: מיתוג cond_compilation_package.vhd
ויחסי ,MA-ו PC רוחבי ,)1–8 KiB) TCM- גודל ה ,)G_MODELSIM)
החלוקה והכפל של ה- PLL.

const_package.vhd תבניות ה- opcode ומסכות ההוראות שבהן משתמש CONTROL, וקידוד
ALU_MUL כולל)EXECUTE-ול CONTROL- המשותף ל ALUOp
שנוסף עבור הרחבת M(.

MUL16.vhd כופל 16×16 ← 32 של חצאי המילה התחתונים של rs2/rs1, הבנוי מארבעה
מכפלים חלקיים בני 8 ביט לפי +) RESULT = P0 +((P1+P2(>>8
(P3>>16), כך ש- Quartus ממפה אותו לכופלים המשובצים.

באמצעות mul מזהה;) combinational decoder) מפענח צירופי CONTROL.vhd
.ALUOp = ALU_MUL ומניע INST_MUL_MASK/INST_MUL

EXECUTE.vhd ה- ALU) חיבור/חיסור, פעולות לוגיות, barrel shifters, השוואות) ולוגיקת
ההסתעפות; יוצר מופע של MUL16 ובוחר את תוצאתו כ- alu_res עבור
.ALU_MUL

ITCM-וה(jalr/jal/branch(הבא PC- מרבב ה ,PC+4 מחבר ,PC- אוגר ה IFETCH.vhd
.(altsyncram)

IDECODE.vhd קובץ אוגרים 32×32, חילוץ שדות ההוראה, כל חמש תבניות ה-
.write-back- והרחבת הסימן, ומרבב ה immediate

. הנכתב על השעון ההפוך ,(altsyncram)DTCM-ה DMEMORY.vhd

aux_package.vhd הצהרות הרכיבים של כל התכן.

ALTPLL PLL.vhd הגוזר את שעון הליבה בן MHz 25 מהמתנד בן MHz 50 שעל הכרטיס; נוצר רק כאשר G_MODELSIM = 0.

RV32IM_CORE.vhd הישות העליונה המבנית; מחווטת את חמשת תת־המודולים וחושפת את אותות
.Signal-Tap- הניפוי ל

טבלה 3: תיאור קבצי ה- HDL של מימוש ה- Single-Cycle RV32IM

9

RTL Viewer תוצאות 3.5
לאחר הסינתזה ב- Quartus בחנו את התכן ב- RTL Viewer כדי לוודא שהמבנה שהתקבל תואם לתרשים המיקרו־ ארכיטקטורה. מכיוון שהתכן העליון מבני, הבלוקים בתצוגה מתאימים אחד־לאחד לבלוקים שבאיור 4.

איור 6: הרמה העליונה של RV32IM_CORE ב- RTL Viewer: חמשת תת־המודולים ואפיקי החיבור ביניהם

RTL Viewer-ב IFETCH איור 7: מודול

10

RTL Viewer-ב IDECODE איור 8: מודול

RTL Viewer-ב CONTROL איור 9: מודול

11

RTL Viewer-ב EXECUTE איור 10: מודול

איור 11: מודול MUL16 ב- RTL Viewer: ארבעת המכפלים החלקיים בני 8 ביט, סכום האמצע והחיבור הסופי

RTL Viewer-ב DMEMORY איור 12: מודול

12

4. חלק 2 - מיקרו־ארכיטקטורת Pipelined RV32IM

בחלק השני של המשימה הרחבנו את מיקרו־ארכיטקטורת ה- single-cycle RV32IM שתוכננה בחלק 1 למימוש pipeline סקלרי בן חמישה שלבים. במקום שהוראה אחת תעבור את כל המסלול הארוך במחזור אחד, המסלול נחתך לחמישה קטעים קצרים המופרדים באוגרי pipeline, וחמש הוראות שונות מתקדמות בו־זמנית, כל אחת בשלב אחר. תדר השעון נקבע כעת לפי השלב האיטי ביותר בלבד ולא לפי סכום כל השלבים.

בהשוואה לתיאור הראשוני של חלק 1 (סעיף 3.2.1), הכופל בן 16 הביט אינו נשאר עוד יחידה אחת בשלב EX: כדי לא להאריך את שלב ה-EX בעומק הלוגי של סכימת שלושת האיברים, הכפל פוצל לשני שלבי pipeline נפרדים.

איור 13: מיקרו־ארכיטקטורת) Pipelined RV32IM(MULDIV partial בת חמישה שלבים

13

4.1 חלוקה לחמישה שלבים ואוגרי ה- pipeline

את חלוקת התכן ביססנו על תת־המודולים הקיימים מחלק 1: כל תת־מודול הפך לשלב pipeline, ואוגר ה- pipeline שבסופו מומש בתוך אותו מודול ולא כישות נפרדת. כך נשמר התכן המבני של הרמה העליונה, המחווטת חמישה שלבים ושתי יחידות בקרת pipeline בדיוק כמו בתרשים. לעומת חלק 1, שלב ה-EX וה- MEM התפצלו כל אחד למחצית מודול הכפל (MULT_1 ו- MULT_2 בהתאמה), ונוסף שלב WB עצמאי שמרכז את מרבב ה- write-back שבחלק 1 היה חלק מ- IDECODE.

שלב מודול VHDL מה מתבצע בשלב אוגר ה- pipeline שבסופו

- מרבב ה ,PC+4 מחבר ,PC- אוגר ה IFETCH.vhd IF 1
ITCM- קריאה מה ,redirect

IF/ID

IDECODE.vhd ID 2
CONTROL.vhd

פענוח, קריאת קובץ האוגרים, ייצור
immediate והרחבת סימן

ID/EX

EXECUTE.vhd EX 3
MULT_1.vhd

,ALU-ה ,forwarding מרבבי
MUL16, מחבר כתובת ההסתעפות,
ארבעת המכפלים החלקיים P0–P3

EX/MEM

DMEMORY.vhd MEM 4
MULT_2.vhd

גישה ל- DTCM, צירוף תוצאות
הכפל, הכרעת הסתעפויות וקפיצות

MEM/WB

:)jal/jalr/PC+4(מרבב תלת כיווני WRITEBACK.vhd WB 5
תוצאת ALU או כפל נתון מה-
DTCM, וכתיבה לקובץ האוגרים

-

טבלה 4: מיפוי חמשת שלבי ה- pipeline למודולי ה- VHDL ולאוגרי ה- pipeline

קובץ האוגרים נקרא בשלב ID ונכתב בשלב WB, ולכן שני הצדדים ממוקמים באותו מודול (IDECODE(. פתרון זה גם מאפשר לממש בתוך המודול את עקיפת הקריאה (read bypass(: כאשר הוראה בשלב WB כותבת לאותו אוגר שהוראה בשלב ID קוראת ממנו, הקריאה מקבלת את הערך החדש (wb_write_data_i (ולא את התוכן הישן שב-RF. בלי עקיפה זו היה נדרש עוד מנגנון forwarding או עוד stall.

14

forwarding- ויחידת ה data hazards 4.2

data hazard מסוג RAW מתרחש כאשר הוראה זקוקה לתוצאה של הוראה קודמת שטרם נכתבה לקובץ האוגרים.
במקום להשהות את ה- pipeline, יחידת ה- forwarding מזרימה את התוצאה ישירות מאוגר ה- pipeline שבו היא כבר
קיימת אל כניסות ה- ALU.

היחידה FORWARD_UNIT.vhd משווה את rs1 ו-rs2 של ההוראה שבשלב EX מול אוגרי היעד של ההוראות שבשלבים MEM ו-WB, ומניעה שני אותות בחירה בני שני ביט) forward_a_o, forward_b_o)המזינים את מרבבי הכניסה של ה- ALU בתוך EXECUTE.vhd:

קידוד שם מקור הנתון מתי

FWD_NONE 00 אוגר ה- ID/EX(הערך שנקרא מקובץ
האוגרים(

אין תלות

של mem_forward_data_o FWD_MEM 10
DMEMORY: תוצאת ה- ALU או תוצאת

שלב 2 של הכפל) MULT_2), לפי
Mul_ctrl_i

תלות במרחק 1

תוצאת)WRITEBACK של write_data_o FWD_WB 01
ALU-או-כפל, נתון DTCM, או PC+4)

תלות במרחק 2

טבלה 5: קידוד מרבבי ה- forwarding ביחידת FORWARD_UNIT

מקור ה- FWD_MEM אינו "שדה ה- ALU של אוגר EX/MEM "כפי שהיה בחלק 1, אלא הפלט הקומבינטורי אינו מכיל את EX/MEM של alu_res- שדה ה mul עבור הוראת .DMEMORY של mem_forward_data_o המכפלה כלל – המכפלה מוכנה רק בשלב MEM לאחר MULT_2 – ולכן ה- forwarding חייב להיות ממקור שכבר ביצע את הבחירה בין ALU לכפל) mem_result_w).

שני תנאים נוספים חייבים להתקיים כדי שהנתון יוזרם: ההוראה הקודמת אכן כותבת לאוגר (' RegWrite = '1(, והיעד
אינו x0. האוגר x0 קשיח לאפס בארכיטקטורת RISC-V, ולכן forwarding אליו היה שגוי. כמו כן נקבעה עדיפות ל-
EX/MEM על פני MEM/WB: אם שתי ההוראות שלפנינו כותבות לאותו אוגר, הערך הנכון הוא של המאוחרת מביניהן, לפי הכלל "הכותב האחרון מנצח".

FORWARD_UNIT יחידת - RTL Viewer :14 איור

15

interlock- ויחידת ה load-use hazard 4.3

forwarding מלא מכסה כמעט את כל ה- data hazards, אך נשאר אחד שאין דרך לכסות: הוראת lw מקבלת את הנתון שלה מה- DTCM רק בסוף שלב MEM, בעוד שההוראה הצמודה אחריה זקוקה לו כבר בתחילת שלב EX שלה, כלומר באותו מחזור עצמו. אין דרך פיזית להזרים נתון אחורה בזמן, ולכן נדרש stall של מחזור אחד.

היחידה HAZARD_UNIT.vhd מממשת בדיקה צירופית טהורה) combinational checkboard), ללא מצב פנימי כלשהו:

stall_o = ex_MemRead_ctrl_i AND (ex_rd_i ≠ x0) AND (ex_rd_i = id_rs1_i OR ex_rd_i = id_rs2_i)

כאשר ' stall_o = '1, הרמה העליונה מפעילה interlock של מחזור אחד:

• המודול IFETCH מקפיא את ה-PC ואת אוגר stall_i (IF/ID), כך שההוראה הצורכת מפוענחת שוב בשלב
ID במחזור הבא.

.)bubble_w = stall_i or flush_i(ID/EX לאוגר bubble מזריק IDECODE המודול •
• הוראת ה- lw עצמה ממשיכה כרגיל במורד ה- pipeline.
מחזור stall אחד מספיק. אחריו ההוראה הצורכת עדיין בשלב ID בעוד ה-lw נמצאת ב- MEM. במחזור שאחריו הצורכת מגיעה ל-EX וה-lw נמצאת ב-WB, ואז נתון הטעינה מוזרם אליה על ידי FORWARD_UNIT בבחירה FWD_WB. שאר המרחקים מכוסים ממילא: צרכן במרחק 2 פוגש את ה-lw ב-WB בלי stall, וצרכן במרחק 3 קורא את קובץ האוגרים ב-ID בדיוק כשה-lw כותבת אליו ב-WB, מקרה שמכוסה על ידי עקיפת הקריאה שבתוך .IDECODE

HAZARD_UNIT יחידת - RTL Viewer :16 איור

16

control hazards 4.4: הכרעת הסתעפויות בשלב MEM

בתכן זה, הסתעפויות מותנות וקפיצות בלתי־מותנות) jalr ,jal)מוכרעות בשלב הרביעי של ה- MEM ,pipeline. בזמן שההוראה המסתעפת מתקדמת בין IF ל- MEM, ה- pipeline ממשיכה לשלוף באופן ספקולטיבי את ההוראות העוקבות בזיכרון. כאשר מתברר שההסתעפות נלקחת, שלוש ההוראות הצעירות ממנה, אלו שבאוגרי ID/EX ,IF/ID ו-EX/MEM, שגויות ויש לבטלן. מכאן שעומק ה- flush הוא 3, וזהו המקדם depth המופיע בנוסחת ה- IPC.

כתובת ה- redirect_addr_w)redirect (נבחר מתוך אוגר ה- EX/MEM: עבור branch ו-jal זוהי תוצאת מחבר כתובת ההסתעפות, ועבור jalr זוהי תוצאת ה- ALU.

ייתכן מצב שבו stall ו- flush פעילים באותו מחזור. במקרה כזה ה- flush גובר: ההוראה שגרמה ל- redirect נמצאת ב-MEM ולכן היא מבוגרת מההוראה שגרמה ל- stall, וממילא ההוראה הצעירה עומדת להתבטל, וה- stall חסר משמעות. כלל קדימות זה מונע איבוד של אירוע redirect.

17

4.5 תוספות ניפוי שגיאות ו- Signal-Tap
לצורך ולידציה וחישוב IPC, הישות העליונה של ה- pipeline מרחיבה את זו של ה- single-cycle בארבע תוספות.

ואוגר נקודת FHCNT ,STCNT ,CLKCNT עם מוני RV32IM_PIPE_CORE - pipeline- איור 17: הישות העליונה של ה
העצירה BPADDR

אות רוחב תפקיד
CLKCNT_o 16 מונה מחזורי שעון מאז האיפוס - המכנה בנוסחת ה- IPC.
STCNT_o 16 מונה stalls ;מתקדם בעליית השעון בכל מחזור שבו ' stall = '1.
מתאפס באיפוס.

FHCNT_o 16 מונה flush ;מתקדם בכל אירוע redirect. מתאפס באיפוס.
BPADDR_i 8 אוגר כתובת נקודת עצירה (ברזולוציית מילה), מוזן מ- SW7–SW0
שעל הכרטיס.
SPTRIGGER_o 1 אות הטריגר ל- Signal-Tap, המופק מהשוואת ה-PC של שלב IF
.BPADDR_i-ל
טבלה 6: אותות הניפוי שנוספו בישות העליונה של ה- pipeline

שתי המשוואות המגדירות את המנגנון:
STRIGGER_o = (IFpc_o[12:2] == BPADDR_i) IPC = [CLKCNT_o − (STCNT_o + 4 + depth · FHCNT_o)] / CLKCNT_o = InstructionCounter / CLKCNT_o , IPC = 1 / CPI איברי המונה מתפרשים כך: מכלל מחזורי השעון מחסירים את מחזורי ה- STCNT_o)stall), את 4 המחזורים של מילוי ה- pipeline בתחילת הריצה (pipeline fill: ההוראה הראשונה זקוקה ל־5 מחזורים כדי לפרוש, ולכן 4 מחזורים אינם מנוצלים), ואת המחזורים שאבדו בכל אירוע redirect, 3 לכל flush. מה שנותר הוא מספר ההוראות שפרשו בפועל.
BPADDR נדגמת מ- SW7–SW0 לתוך אוגר bpaddr_q בכל עליית שעון; KEY0 משמש כאיפוס (rst_i(.

4.6 תיאור קבצי ה- HDL

18

קובץ תיאור

cond_compilation_package.vhd,

const_package.vhd, PLL.vhd זהים בתפקידם לקבצים המקבילים בחלק 1.

aux_package.vhd

הצהרות הרכיבים של כל שלבי ה- pipeline) כולל , MULT_1, MULT_2
והישות(WRITEBACK, HAZARD_UNIT, FORWARD_UNIT .RV32IM_PIPE_CORE

ומפנה bubble מזריק flush_i ,) הקפאה)stall_i מקבל כניסת PC-ה .IF/ID ואוגר IF שלב IFETCH.vhd
.STRIGGER משמש להשוואת if_pc_o-ו ,redirect- ליעד ה PC- את ה

IDECODE.vhd

שלב ID ואוגר ID/EX) נתונים, immediate, rs1/rs2/rd ואותות בקרה). קובץ האוגרים
נכתב מ- wb_write_data_i) פלט WRITEBACK (בעוד הקריאות מתבצעות ב-ID, כולל
עקיפת קריאה לאותו מחזור.

CONTROL.vhd אותו מפענח צירופי מחלק 1, היושב בשלב ID ;פלטיו נעים במורד ה- pipeline יחד עם

ההוראה.

MULT_1 ומופע ,ALU- על שתי כניסות ה forwarding מרבבי .EX/MEM ואוגר EX שלב EXECUTE.vhd
(ארבעת המכפלים החלקיים P0–P3 מהאופרנדים המוזרמים).

MULT_1.vhd שלב 1 של הכפל (EX, מופע MUL1(: ארבעה מכפלים חלקיים 8×8 – מ-16 הביטים

התחתונים של האופרנדים המוזרמים.

DMEMORY.vhd

שלב MEM ואוגר MEM/WB. מכיל את ה- DTCM ואת מופע MULT_2 ;מרבב
mem_result_w בוחר בין תוצאת ALU לתוצאת הכפל, ומזין הן את MEM/WB והן את
.mem_forward_data_o

MULT_2.vhd שלב 2 של הכפל (MEM, מופע MUL2(: מצרף את P0–P3 שעברו דרך EX/MEM ל-

.res_o

WRITEBACK.vhd

/ או-כפל-ALU תוצאת / PC+4) jal/jalr (:)WB מופע (WB של שלב write-back- מרבב ה
נתון DTCM. הפלט (write_data_o (מזין את פורט הכתיבה של קובץ האוגרים, את עקיפת
הקריאה ב-ID, ואת FWD_WB.

,EX מול אוגר היעד שבשלב ID של שלב rs1/rs2 משווה את :load-use- צירופי ל interlock HAZARD_UNIT.vhd

ומשהה מחזור אחד.

FORWARD_UNIT.vhd משווה את rs1/rs2 של שלב EX מול אוגרי היעד שב- EX/MEM וב- MEM/WB) בתנאי

RegWrite='1 'ו- rd≠x0 (ומניע את אותות בחירת מרבבי ה- forwarding.

RV32IM_PIPE_CORE.vhd הישות העליונה המבנית; הסתעפויות וקפיצות מוכרעות ב- flush (MEM בעומק 3). כוללת את

.STRIGGER והשוואת BPADDR אוגר ,CLKCNT/STCNT/FHCNT מוני

טבלה 7: תיאור קבצי ה- HDL של מימוש ה- pipeline

19

RTL Viewer תוצאות 4.7

Netlist Navigator: control:CTL, - תשעה מופעים ב .RV32IM_PIPE_CORE הרמה העליונה של – RTL Viewer :19 איור dmemory:MEM, Execute:EXE, FORWARD_UNIT:FWD, HAZARD_UNIT:HZD, Idecode:ID, Ifetch:IFE, writeback:WB, PLL:\G0:MCLK

IF שלב - RTL Viewer :21 איור

21

ID בשלב CONTROL מודול - RTL Viewer :23 איור

23

MEM שלב - RTL Viewer :25 איור

WB שלב - RTL Viewer :26 איור

24

איור RTL Viewer – multiplier_1:MUL1 :27(שלב EX): ארבעת המכפלים החלקיים 8×8

)MEM שלב)RTL Viewer – multiplier_2:MUL2 :25 איור

25

5. וריפיקציה וולידציה של התכן

5.1 מתודולוגיית האימות

האימות בוצע לפי זרימת התכן שבסעיף 8 של המשימה, עם RARS כמודל זהב. עבור כל אפליקציית benchmark, קובץ ה- DTCM.h שמייצר RARS בסוף הריצה משמש כתוצאה הנכונה, ומולו מושווה קובץ ה- DTCM.mem שנשפך מהסימולציה ב- ModelSim, וכן קובץ ה- DTCM.hex שמפיק ISMCE מהריצה על ה- FPGA.

אפליקציות ה- benchmark מסתיימות בלולאה אינסופית (הוראת beq x0,x0,0 בקידוד 00000063). במימוש ה-single-cycle מספיק לזהות הוראה זו בשלב הפענוח כדי לעצור, אבל ב- pipeline זיהוי כזה שגוי: ההסתעפויות מוכרעות ב- MEM, ולכן השליפה הספקולטיבית מביאה את הוראת הסיום לשלב ID כבר באיטרציה הראשונה של לולאה כלשהי, ועצירה על כך הייתה מפסיקה את הריצה מוקדם מדי. לכן נעשה שימוש בקריטריון אחר: אירוע flush שיעד ההפניה שלו שווה ל- PC+4 של ההוראה המפנה עצמה (target+4 == mem_pc_plus4(. תנאי כזה יכול להתקיים רק עבור קפיצה עצמית, ולכן הוא חד־משמעי. במימוש ה- single-cycle העצירה נעשית פשוט על ההוראה המפוענחת 00000063.

5.2 תוצאת ההשוואה מול מודל הזהב

כל ארבע האפליקציות הורצו בשני המימושים. תמונות זיכרון הנתונים שנשפכו נבדקו הן מול DTCM.h של RARS והן זו מול זו.

ההשוואה הישירה בין DTCM_testN.mem של ה- single-cycle לבין זה של ה- pipeline העלתה זהות מוחלטת בכל 1024 המילים ובכל ארבעת הטסטים. מכאן ששני המימושים שקולים פונקציונלית: מנגנוני ה- , pipeline – forwarding interlock ל- load-use, ביטול הוראות ספקולטיביות והכרעת הסתעפויות ב- MEM, וכן פיצול הכפל לשני שלבים – אינם משנים ולו ביט אחד בתוצאה הסופית של התוכנית.

26

5.3 ניתוח דיאגרמות גלים

מחזור השעון ב- testbench הוא ns 100, ולכן זמן הסימולציה מתורגם ישירות למחזורים. על בסיס זה ניתחנו את דיאגרמות הגלים של ארבע האפליקציות בשני המימושים ואת התנהגות התזמון של המערכת.

לכל בדיקה תועדו: תוצאת מונה ההוראות של RARS, תחילת הריצה ב- ModelSim ובחומרה (Signal-Tap על ה-FPGA (עבור שני המימושים, וסוף הריצה.

:test1 5.3.1

test1 עבור RARS Instruction Counter :27 איור

Single Cycle

Single Cycle סוף הריצה ,(Single Cycle(test1 :28 איור

מספר המחזורים (134) שווה למספר ההוראות שבוצעו, כצפוי מ- IPC=1) גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).

27

ModelSim- תחילת ריצה ב,)Single Cycle (test1 :29 איור

איור Single Cycle (test1 :30), אותה תחילת ריצה כפי שנצפתה ב- Signal-Tap
אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

Pipeline

ModelSim- תחילת ריצה ב ,)Pipeline (test1 :31 איור

איור Pipeline (test1 :32), אותה תחילת ריצה כפי שנצפתה ב- Signal-Tap
אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

איור Pipeline (test1 :33), פעולת הכופל המפוצל ב- ModelSim

;p1_o=p3_o=0000 ,p0_o=0100, p2_o=0004 מניבים a_i=0140, b_i=0004

במחזור הבא, mul_res_w=00000500 = 0x140 × 4 — ההוכחה הישירה לכך שהכפל אכן מתפרש על שני מחזורי נפרדים pipeline

28

איור Pipeline (test1 :34), סוף הריצה

CLKCNT_o גדול ממספר ההוראות בשל תקורת ה- stalls/flush ;ההשוואה המלאה מול ה- IPC התיאורטי מופיעה בטבלה 15 שבסעיף 7.

29

:test2 5.3.2

test2 עבור RARS Instruction Counter :35 איור

Single Cycle

סוף הריצה ,)Single Cycle (test2 :38 איור

מספר המחזורים (1514) שווה למספר ההוראות שבוצעו, כצפוי מ- IPC=1) גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).

30

תחילת ריצה ,)Single Cycle (test2 :36 איור

Signal-Tap- תחילת ריצה כפי שנצפתה ב ,)Single Cycle (test2 :37 איור

אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

Pipeline

איור Pipeline (test2 :41), סוף הריצה

CLKCNT_o גדול ממספר ההוראות בשל תקורת ה- stalls/flush ;ההשוואה המלאה מול ה- IPC התיאורטי מופיעה בטבלה 15 שבסעיף 7.

31

pipeline- תחילת ריצה ב ,)Pipeline (test2 :39 איור

איור Pipeline (test2 :40), תחילת ריצה כפי שנצפתה ב- Signal-Tap

אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

32

test3 5.3.3: לולאות והסתעפויות נלקחות

test3 עבור RARS Instruction Counter :42 איור

Single Cycle

סוף הריצה ,)Single Cycle (test3 :45 איור

מספר המחזורים (2725) שווה למספר ההוראות שבוצעו, כצפוי מ- IPC=1) גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).

33

תחילת ריצה ,)Single Cycle (test3 :43 איור

Signal-Tap- תחילת ריצה כפי שנצפתה ב ,)Single Cycle (test3 :44 איור

אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

Pipeline

איור Pipeline (test3 :48), סוף הריצה

CLKCNT_o גדול ממספר ההוראות בשל תקורת ה- stalls/flush ;ההשוואה המלאה מול ה- IPC התיאורטי מופיעה בטבלה 15 שבסעיף 7.

34

איור Pipeline (test3 :46), תחילת ריצה

איור Pipeline (test3 :47), תחילת ריצה כפי שנצפתה ב- Signal-Tap

אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

35

test4 5.3.4: קפיצות בלתי־מותנות וקריאות לפונקציות

test4 עבור RARS Instruction Counter :49 איור

Single Cycle

סוף הריצה ,)Single Cycle (test4 :52 איור

מספר המחזורים (2735) שווה למספר ההוראות שבוצעו, כצפוי מ- IPC=1) גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).

36

תחילת ריצה ,)Single Cycle (test4 :50 איור

Signal-Tap- כפי שנצפתה ב jalr אותה קפיצת ,)Single Cycle (test4 :51 איור

אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

Pipeline

איור Pipeline (test4 :55), סוף הריצה

CLKCNT_o גדול ממספר ההוראות בשל תקורת ה- stalls/flush ;ההשוואה המלאה מול ה- IPC התיאורטי מופיעה בטבלה 15 שבסעיף 7.

37

pipeline- תחילת ריצה ב ,)Pipeline (test4 :53 איור

איור Pipeline (test4 :54), תחילת ריצה כפי שנצפתה ב- Signal-Tap

אותו רצף הקסדצימלי של הוראות כמו ב- ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

נקודות ניתוח משותפות לארבעת הטסטים:

• קריאת הדיאגרמות שונה מהותית בין המימושים: ב- Single-Cycle כל שורות האותות מתייחסות לאותה
הוראה בכל רגע נתון, ואילו ב- pipeline כל אחד מחמשת הזוגות) IFpc_o…WBpc_o)מתייחס לשלב אחר
– כך שבכל צילום מסך נראות חמש הוראות שונות בו־זמנית.

• בכל הטסטים, תחילת הריצה ב- ModelSim ובחומרה) Signal-Tap)מציגות את אותו רצף הוראות
הקסדצימלי – אימות עקבי לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה.

• test2 הוא הטסט היחיד עם STCNT גבוה) test3 ;(100 ו- test4 הם היחידים עם FHCNT גבוה ו-
STCNT=0 – שני הטסטים יחד מכסים את שני סוגי ה- hazards שה- pipeline נדרש להתמודד איתם.

• סוף כל טסט מזוהה ב- Single-Cycle בהוראה beq x0,x0,0) 00000063)עם brTaken_o=1, וב-
pipeline באירועי flush_w תקופתיים הנובעים מאותה קפיצה עצמית.

38

6. ניתוח PPA

6.1 שלוש טבלאות ה- PPA

שלוש טבלאות אפיון ה- PPA הנדרשות בסעיף 7 של הגדרת המשימה מרוכזות כאן יחד. צילומי דוחות ה- Quartus שמהם נלקחו הערכים, והניתוח המפורט של כל מדד, מופיעים בסעיפים 6.2–6.4.

Area

Logic
elements

Registers I/O pins Embedded
memory bits

Embedded 9-bit

Multipliers

PLLs

Single-Cycle
RV32IM

3,120 1,279 270 131,072 4 1

Pipelined
RV32IM

3,538 1,877 284 131,072 4 1

טבלה 10: צריכת המשאבים של שני המימושים

Performance

fmax Critical path (what is the slowest submodule and
why does it cause the critical path)

fMCLK (= fsysclk)

Single-Cycle
RV32IM

המסלול של sw/lw: מה- ITCM, דרך מרבבי ה- IDECODE וה- MHz 26.81 ALU, עד אוגר הכתובת של ה- DTCM. הכתובת היא תוצאת ה- ALU,
ולכן כל השרשרת חייבת להסתיים לפני הגישה לזיכרון.

25 MHz

Pipelined
RV32IM

51.7 MHz

EX/MEM מאוגר :EX-ב forwarding המסלול של
דרך יחידת ,(MEM-ב rd[0] שהוא ,ex_mem_instruction_q[7](
ה- forwarding, מרבבי האופרנדים וה- ALU, עד
] ex_mem_alu_res_q[15. תוצאת ה- MEM מוזרמת ל-EX באותו
מחזור, ולכן כל השרשרת חייבת להסתיים לפני דפדוף EX/MEM.

25 MHz

טבלה 11: תדר מרבי, מסלול קריטי ותדר העבודה בפועל

Power

Total power
consumption

Static power
consumption

Dynamic power
consumption

I/O power
consumption

Single-Cycle
RV32IM

275.98 mW 102.39 mW 22.93 mW 150.67 mW

Pipelined
RV32IM

292.21 mW 102.49 mW 31.81 mW 157.91 mW

טבלה 12: צריכת ההספק של שני המימושים

39

Area 6.2: צריכת משאבים

single-cycle- במימוש ה RV32IM_CORE של Fitter Summary איור 42: דוח

pipeline- במימוש ה RV32IM_PIPE_CORE של Fitter Summary איור 43: דוח

40

ניתוח

• מספר האוגרים גדל ל- 1,877)+% 46.8 לעומת ה- Single-Cycle), הגידול הגדול ביותר מבין המדדים וזהו
חייבים pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB) ארבעה אוגרי .pipeline- המחיר הישיר של ה
לשמור בכל מחזור את ההוראה, ה-PC, שני האופרנדים, ה- immediate, מזהי האוגרים וכל אותות הבקרה, וכעת גם ארבע תוצאות ביניים בנות 16 ביט של הכפל) P0–P3)באוגר EX/MEM. נוספים להם שלושת מוני הניפוי בני 16 הביט ואוגר BPADDR.

• • הלוגיקה גדלה ל- Logic elements 3,538 – עלייה של % 13.4 בלבד, נמוכה משמעותית מהגידול באוגרים. מסלול הנתונים עצמו לא הוכפל; נוספו , FORWARD_UNIT, HAZARD_UNIT מרבבי ה- forwarding, לוגיקת ה- redirect, מודול WRITEBACK ומחבר הצירוף שב- MULT_2.

• כמו ב- Single-Cycle, נוצלו 4 כופלים משובצים – פיצול המכפל היא לוגיקת חיבור בלבד, ואינה צורכת כופלים
נוספים.

• הגידול של +14 פינים)284→270(נובע מאותות הניפוי של ,[Figure 8: CLKCNT_o]15:0
- וחמשת זוגות ה ,STCNT_o[15:0], FHCNT_o[15:0], BPADDR_i[7:0], STRIGGER_o הם אותות פנימיים בלבד ואינם flush_w-ו PC/instruction (IFpc_o…WBinstruction_o). stall_w יוצאים כפינים עליונים.

41

Performance 6.3: תדר מרבי ומסלול קריטי

MCLK על שעון הליבה 26.81 MHz :single-cycle- של מימוש ה Fmax Summary איור 44: דוח

איור 45: דוח Fmax Summary של מימוש ה- MHz :pipeline 51.7 על שעון הליבה MCLK

42

אל Ifetch:IFE|...|ram_block|porta_address_reg-מ ,single-cycle- איור 46: המסלול הקריטי של מימוש ה
ולכן זהו מסלול חצי־מחזור ,INVERTED מסומן Latch Clock-ה .dmemory:MEM|...|porta_address_reg

חישוב ה- fmax מתוך המסלול
השהיית המסלול היא ns 18.555 בלבד, ולכאורה הייתה מאפשרת MHz 53≈. אלא שכפי שהוסבר בסעיף 3.1, ה-DTCM נכתב על השעון ההפוך, וזה נראה מפורשות בשדה Latch Clock שבדוח, המסומן INVERTED. לכן המסלול כולו חייב להסתיים בתוך חצי מחזור שעון:
fmax = 1 / (2 · tpath) = 1 / (2 · 18.65 ns) ≈ 26.8 MHz

החישוב מתאים ל־ MHz 26.81 שדיווח ה- Timing Analyzer, ומכאן שזיהינו נכון את המסלול.

43

איור 47: דוח Report Timing של ה- pipeline: עשרת המסלולים בעלי ה- slack הקטן ביותר: כולם מסלולי DTCM
14.459 מקסימלי בטבלה זו ram_block; slack אל dmemory:MEM-מ) mem_wb_dtcm_data_q)

זיהוי המסלול הקריטי של ה- pipeline

זיהוי המסלול הקובע את fmax ב- pipeline: כמו ב- Single-Cycle, ה- DTCM ב- pipeline עדיין נכתבת על השעון
ההפוך (DMEMORY.vhd, wrclk_w <= NOT clk_i(, כך שבתכן מתקיימים בו־זמנית מסלולי חצי־מחזור (לתוך
אוגרי ה- DTCM (ומסלולי מחזור־מלא (בין אוגרי ה- pipeline(. דרישת הזמן של הראשונים משתנה כמו T/2 ושל
האחרונים כמו T, ולכן ה- slack שמדווח Report Timing עבור שני הסוגים אינו בר-השוואה ישירה. עשרת המסלולים
המוצגים באיור 33 הם כולם מסוג slack (DTCM עד ns 14.459, השהיית נתונים טהורה של ns 5.112 בלבד עבור
המסלול הראשון), כלומר fmax_DTCM ≈ 1/(2·5.112 ns)97.8 ≈ MHz — הרבה מעל MHz 51.7. משמעות
הדבר: המסלול המדווח כבעל ה- slack הקטן ביותר אינו המסלול שקובע בפועל את MHz 51.7.

סוג המסלול השהיה תדר מרבי שהמסלול מתיר

EX/MEM → כתובת ה- DTCM(חצי־מחזור, מהדוח(MHz 5.112 ns 97.8 ≈ (5.112 · 2) / 1
המסלול הקובע בפועל (מחזור־מלא) MHz 19.34 ns 51.7 ≈ 19.34 / 1
טבלה 14: השהיית מסלול חצי־המחזור ומסלול המחזור־המלא ב- pipeline והתדר שכל אחד מהם מתיר; מכאן שהמסלול בעל ה- slack
הקטן ביותר אינו זה שקובע את fmax

כלומר fmax אינו מוגבל עוד על ידי ה- DTCM) כפי שהיה ב- Single-Cycle (אלא על ידי מסלול מחזור־מלא כלשהו
בין שני אוגרי pipeline סמוכים, שאורכו כ- ns 19.34. הדוח שברשותנו אינו חושף במפורש את זהות המסלול הזה (עשרת המסלולים הגרועים ביותר התבררו כשייכים ליחס השעון החצי־מחזורי, שלו slack חיובי גדול יותר באופן יחסי
לתקופתו הקצרה יותר). מבחינה מבנית, המועמד הסביר ביותר להיות שלב ה- EX/ID: שלב ה-ID עדיין מרכז את
קריאת קובץ האוגרים 32×32, את הפענוח המלא ב- CONTROL, את ייצור חמש תבניות ה- immediate ואת בדיקת
ה- load-use ב- HAZARD_UNIT – בדיוק כפי שהיה קודם. לעומת זאת שלב ה-EX נעשה קליל יותר משמעותית
ביחס למה שהיה ב- Single-Cycle: הוא מכיל כעת רק את ה- ALU ואת ארבעת המכפלים החלקיים המהירים של
MULT_1) מבוססי כופל חומרתי), ולא עוד את שרשרת הסכימה המלאה של MUL16. זהו כיוון הסבר מבני סביר
לתוצאת ה- fmax היחסית הגבוהה שהתקבלה (MHz 51.7(: פיצול הכפל בין MULT_1 ל- MULT_2 הקל את שלב
ה- EX ביחס למצב שבו כל שרשרת הכפל הייתה יושבת בשלב יחיד.

44

זיהוי ישיר של המסלול הקריטי בפועל של ה- pipeline

איור 48: דוח Report Timing: המסלול הראשון והגרוע ביותר) Slack=20.656 ns)הוא Execute:EX|ex_mem_instruction_q[7] →Execute:EX|ex_mem_alu_res_q[15]

נתוני המסלול הקובע, כפי שדווחו על ידי הכלי: Data Delay = 19.209 ns, Data Arrival Time = 19.496 ns .Slack = 20.656 ns-ו Data Required Time = 40.152 ns

מכאן שהתדר המרבי שהמסלול הזה מתיר הוא fmax = 1/19.344 ns ≈ 51.7 MHz – התאמה מדויקת לערך שדיווח דוח ה- Fmax Summary שהוצג קודם בסעיף זה, המאשרת שזהו אכן המסלול הקובע בפועל את fmax של ה-.pipeline

שני קצות המסלול) ex_mem_instruction_q]7 [ו- ex_mem_alu_res_q]15 [)הם שני שדות של אותו אוגר עצמו
– אוגר ה- EX/MEM. כלומר המסלול הקריטי בפועל אינו חוצה גבול שלם בין שני שלבי pipeline, אלא הוא מסלול קומבינטורי הפנימי ללוגיקת שלב ה-EX שמזינה את כניסת ה-D של אוגר ה- EX/MEM(ככל הנראה שרשרת הבחירה/מרבוב בין תוצאת ה- ALU לתוצאת MULT_1, המושפעת גם מסיביות של ההוראה הממתינה כבר באוגר(. ממצא זה מדייק את ההשערה המבנית שהוצגה לעיל: המסלול הקובע יושב בפועל בשלב ה-EX(לא ב-ID כפי שהוצע כמועמד(, ומעורבת בו ישירות לוגיקת הבחירה בין ה- ALU ל- MULT_1.

45

השוואה לערך התיאורטי

לפי המשימה, תכן pipeline איכותי אמור להתקרב ל־

fmax

pipeline
= 5 · fmax

single-cycle

= 5 · 26.81 = 134.05 MHz

בפועל התקבלו MHz 51.7, כלומר שיפור של × 1.56 בלבד ביחס ל- single-cycle. הפער נובע מארבע סיבות מבניות:

1. ההנחה שמאחורי הכפולה ×5 היא שניתן לחלק את המסלול לחמישה חלקים שווים. בפועל, השהיית הגישה לבלוקי ה- M9K היא רכיב קשיח שאינו ניתן לחיתוך על ידי הוספת אוגרים, והיא מהווה רצפת השהיה ששלבי
ה-IF וה- MEM אינם יכולים לרדת מתחתיה.

2. הכתיבה על השעון ההפוך נשמרה גם ב- pipeline, ולכן המסלול מאוגר ה- EX/MEM אל כתובת ה- DTCM נותר מסלול חצי־מחזור. הוא לבדו כבר מגביל את התכן ל־ MHz 97.8≈, הרבה מתחת ל־ MHz 134

התיאורטיים.

3. השלבים אינם מאוזנים ביניהם. שלב ה-ID מרכז קריאה מקובץ אוגרים 32×32, פענוח מלא, ייצור חמש תבניות immediate ובדיקת hazards ;זהו כנראה השלב האיטי שקובע את התדר.

4. ל- pipeline עצמו יש תקורה: כל שלב משלם clock-to-Q של אוגר ה- pipeline ו- setup של האוגר הבא, ובנוסף אי־ודאות השעון. תקורה זו אינה מתחלקת ב־5.

46

Power 6.4: צריכת הספק

single-cycle- של מימוש ה Power Analyzer Summary איור 48: דוח

pipeline- של מימוש ה Power Analyzer Summary איור 49: דוח

47

7. ניתוח IPC על בסיס אפליקציות benchmark

סעיף c.iii.8 של המשימה דורש לאמת את ה- IPC בהשוואה בין שני אגפי המשוואה שבסעיף 4.5. אגף אחד מחושב ממוני החומרה, והאגף השני הוא מספר ההוראות שהאפליקציה מבצעת בפועל. כדי שההשוואה תהיה משמעותית, מספר ההוראות הזה צריך להיות ידוע במדויק.

7.1 מקור בלתי־תלוי למספר ההוראות

לשם כך נעזרנו במאפיין שתואר בסעיף 3.3. במימוש ה- single-cycle מתקיים IPC = 1 בהגדרה, ולכן מונה מחזורי
השעון שלו בסוף הריצה שווה בדיוק למספר ההוראות שהאפליקציה מבצעת. שני המימושים מריצים את אותו קוד בינארי ומייצרים DTCM זהה (סעיף 5.2), ולכן מונה המחזורים שלו משמש לנו מודל זהב לבדיקת מונה ההוראות של .pipeline-ה

single-cycle מחזורי benchmark
(= מספר ההוראות)

הוראות שפרשו FHCNT STCNT CLKCNT

CLKCNT−(STCNT+4+3·FH
CNT)

IPC

0.807 134 6 10 166 134 test1

0.7905 1,513 100 100 1,914 1,514 test2
0.7519 2,722 298 0 3,620 2,725 test3
0.7489 2,732 304 0 3,648 2,735 test4
טבלה 15: מוני החומרה וחישוב ה- IPC לאפליקציות ה- benchmark

אימות המשוואה

בכל ארבעת הטסטים מספר ההוראות שהתקבל מהמונים קטן ממודל הזהב בדיוק ב-3, למשל ב- test1: − 167 131 =)8 ∙ 3 + 4 + 8(, לעומת 134 במודל הזהב, ועבור test3: = 2722(298 ∙ 3 + 4 + 0)− 3620, לעומת 2,725. ההפרש הקבוע של depth = 3 בדיוק, בכל ארבעת הטסטים ללא יוצא מן הכלל, מצביע על מקור שיטתי ולא על שגיאת מדידה אקראית. ה- flush ה"אחרון" שבצילום המסך הוא כבר האיטרציה הראשונה של לולאת הסיום
האינסופית, לא אירוע flush אמיתי בגוף התוכנית, ונוסחת ה- IPC" מקנסת" אותו פי depth=3 מבלי שהוא תרם הוראות
ממשיות. בהתעלם מהפרש שיטתי וקבוע זה, ההתאמה בין שני אגפי המשוואה מצוינת ומאמתת הן את המונים והן את
הקבועים 4 (מילוי ה- pipeline (ו-3 (עומק ה- flush (שבנוסחה.

48

7.2 זמן ריצה בפועל
ערך IPC קטן מ־1 אינו מעיד שה- pipeline נחותה; הוא מתאר את יעילות המחזורים בלבד. זמן הריצה בפועל הוא מכפלה של מספר המחזורים בתקופת השעון:

Texec = Cycles × Tclk = Cycles / f

שיפור 19.34 ns × מחזורים :pipeline 37.30 ns × מחזורים :SC benchmark
1.55× 167→3.23us 134→5us test1

1.52× 1915→37.04us 1514→56.47us test2

1.45× 3620→70.01us 2725→101.64us test3

1.45× 3648→70.56us 2735→102.02us test4

טבלה 16: זמן ריצה בפועל כאשר כל מימוש פועל בתדר המרבי שלו (26.81 ו־MHz 51.7 בהתאמה)

השיפור בפועל, פי 1.45 עד פי 1.55, נמוך משיפור התדר לבדו (פי 1.93) משום שה IPC-של ה pipeline-קטן מ-1 : speedup = fmax_ratio × IPC_pipeline; פורמלית .IPC=1 תמיד Single-Cycle- כ-0.75–0.79), בעוד ש) עבור test2 למשל 1.93 × 0.789 ≈ 1.52, בהתאמה טובה לערך שחושב ישירות מהמחזורים.

בתצורה הנוכחית, שני המימושים מוגדרים לאותו יחס) PLL(G_PLL_DIV=2, G_PLL_MUL=1, ולכן בפועל שניהם עובדים ב- MHz 25 בלבד. בתדר זהה זמן הריצה נקבע לפי מספר המחזורים בלבד, ולכן ה- pipeline יוצא דווקא .)Single-Cycle-ב MHz = 60.56 µs 1,514/25 מול test2: 1,915/25 MHz = 76.60 µs- איטי יותר(למשל ב כדי לממש את הפוטנציאל שחושב לעיל יש להעלות את יחס ה- PLL של ה- pipeline. מאחר שנמדד fmax של 51.7 MHz, קיים headroom משמעותי מעל תדר העבודה הנוכחי (MHz 25(: ניתן להשתמש למשל ביחס הפשוט (G_PLL_MUL=1, G_PLL_DIV=1(50 MHz – כמעט כפול מתדר העבודה הנוכחי, ועדיין מתחת ל- fmax
הנמדד.

49

8. סיכום ומסקנות

בעבודה זו תכננו שני מימושים של מעבד RV32IM תואם RISC-V, סינתזנו אותם ואימתנו את פעולתם. המימוש
load--ל interlock ,מלא forwarding סקלרי בן חמישה שלבים עם pipeline והשני ,single-cycle הראשון הוא
use, הכרעת הסתעפויות ב- MEM, וכופל בן 16 ביט המפוצל לשני שלבים.

תוצאות עיקריות

• ארבע אפליקציות ה- benchmark הופקו בשני המימושים, ותמונות ה- DTCM שהתקבלו זהות למודל הזהב
של RARS, וגם זהות בית־בבית זו לזו. שני המעבדים נכונים אפוא מבחינה פונקציונלית, כולל פיצול הכפל
לשני שלבים.

• הכופל מופה כמצופה: ארבעה כופלים משובצים בני 9 ביט נוצלו בפועל בשני המימושים (ב- Single-Cycle
בתוך MUL16, וב- pipeline בתוך MULT_1(, בהתאם לאלגוריתם הנדרש.

• נוסחת ה- IPC אומתה על ארבעת הטסטים: מונה ההוראות המחושב מהמונים קטן ממספר ההוראות שנמדד ב-
Single-Cycle בהפרש קבוע של depth=) 3 (בכל ארבעת הטסטים – פער שיטתי הנובע מרגע העצירה בקצה לולאת הסיום (סעיף 7.2), ולא משגיאה אקראית.

מסקנות PPA

• ה- pipeline קונה ביצועים באוגרים: +% 46.8 אוגרים לעומת +% 13.4 לוגיקה בלבד. כך נראית המרה של
השהיה לתפוקה.

• מחיר ההספק (+%5.9 סך הכול, +% 38.7 בהספק הדינמי) קטן משמעותית ביחס לשיפור התדר (פי 1.93),
ולכן יחס הביצועים־להספק השתפר.

• פיצול הכפל לשני שלבים (MULT_1/MULT_2 (לא רק תיקן את הארכיטקטורה להתאמה לאיור 7 של
המשימה, אלא גם הקל את שלב ה-EX והעלה את fmax) מ-מסלול מלא הכולל MUL16 ב-EX, אל שלב EX קל יותר עם ארבעה מכפלים חלקיים בלבד).

• הגישה ל- fmax_SC×5 (MHz 135 (עדיין רחוקה (הושג כ-% 38.6 מהתיאורטי): רצפת ההשהיה של
הזיכרון המשובץ, אי־איזון בין השלבים (ID עדיין הכבד ביותר), והתקורה המובנית של אוגרי ה- pipeline ממשיכים להגביל.